LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 07 – ABSTRACT & INTERFACE



NAMA : Khairun Fahmi

NIM : 103132400035

KELAS : S1SD–05-A

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Kelas abstrak merupakan kelas yang digunakan untuk merepresentasikan konsep umum suatu entitas dengan kata kunci abstract, di mana kelas ini tidak dapat diinstansiasi secara langsung menjadi objek, dapat memuat variabel biasa maupun konstruktor, serta berfungsi sebagai kelas induk yang mewajibkan *subclass*-nya untuk mengimplementasikan *abstract method* yang dideklarasikannya. Berbeda dengan kelas abstrak, *interface* bertindak sebagai kontrak yang mendefinisikan method dan konstanta (public static final) tanpa konstruktor, serta mendukung pewarisan jamak (*multiple inheritance*) melalui kata kunci implements. Sejak pembaruan Java 8, *interface* juga diperluas sehingga mampu memiliki method berbadan program melalui *default method* yang menjaga kompatibilitas kode lama tanpa mewajibkan *override*, serta *static method* yang dapat dipanggil langsung melalui nama *interface*-nya.

B. Guided

1. Abstract Class

Employee

```
package com.mycompany.guidedmodul07;

public abstract class Employee {
    private String firstName;
    private String lastName;
    private String socialSecurityNumber;

    // three-argument constructor
    public Employee(String first, String last, String ssn) {
        this.firstName = first;
        this.lastName = last;
        this.socialSecurityNumber = ssn;
    }

    public void setFirstName(String first) {
        this.firstName = first;
    }

    public String getFirstName() {
        return firstName;
    }

    public void setLastName(String last) {
        this.lastName = last;
    }

    public String getLastName() {
        return lastName;
```

```
    }

    public void setSocialSecurityNumber(String ssn) {
       this.socialSecurityNumber = ssn;
    }

    public String getSocialSecurityNumber() {
       return socialSecurityNumber;
    }

    @Override
    public String toString() {
       return String.format("%s %s\nsocial security number: %s",
             getFirstName(), getLastName(), getSocialSecurityNumber());
    }

    // abstract method yang wajib dioverride oleh subclass
    public abstract double earnings();
}
```

SalariedEmployee

```
package com.mycompany.guidedmodul07;

public class SalariedEmployee extends Employee {
   private double weeklySalary;

   // four-argument constructor
   public SalariedEmployee(String first, String last, String ssn, double salary) {
      super(first, last, ssn);
      setWeeklySalary(salary);
   }

   public void setWeeklySalary(double salary) {
      this.weeklySalary = (salary < 0.0) ? 0.0 : salary;
   }

   public double getWeeklySalary() {
      return weeklySalary;
   }

   @Override
   public double earnings() {
      return getWeeklySalary();
   }

   @Override
   public String toString() {
      return String.format("salaried employee: %s\n%s: $%,.2f",
            super.toString(), "weekly salary", getWeeklySalary());
   }
```

```
        }
```

Output :

```
==========================================
1. PENGUJIAN ABSTRACT CLASS (EMPLOYEE)
==========================================
salaried employee: John Smith
social security number: 111-11-1111
weekly salary: $800,00
Total Earnings: $800,00

==========================================
2. PENGUJIAN INTERFACE (PAYABLE & INVOICE)
==========================================
invoice:
part number: 01234 (Seat)
quantity: 2
price per item: $375,00
Total Payment: $750,00
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

2. Interface Dasar

Payable

```
package com.mycompany.guidedmodul07;

public interface Payable {
   double getPaymentAmount();
}
```

Invoice

```
package com.mycompany.guidedmodul07;

public class Invoice implements Payable {
   private String partNumber;
   private String partDescription;
   private int quantity;
   private double pricePerItem;

   // four-argument constructor
   public Invoice(String part, String description, int count, double price) {
      this.partNumber = part;
```

```
        this.partDescription = description;
        setQuantity(count);
        setPricePerItem(price);
    }

    public void setPartNumber(String part) {
        this.partNumber = part;
    }

    public String getPartNumber() {
        return partNumber;
    }

    public void setPartDescription(String description) {
        this.partDescription = description;
    }

    public String getPartDescription() {
        return partDescription;
    }

    public void setQuantity(int count) {
        this.quantity = (count < 0) ? 0 : count;
    }

    public int getQuantity() {
        return quantity;
    }

    public void setPricePerItem(double price) {
        this.pricePerItem = (price < 0.0) ? 0.0 : price;
    }

    public double getPricePerItem() {
        return pricePerItem;
    }

    @Override
    public String toString() {
        return String.format("%s: \n%s: %s (%s) \n%s: %d \n%s: $%,.2f",
                "invoice", "part number", getPartNumber(), getPartDescription(),
                "quantity", getQuantity(), "price per item", getPricePerItem());
    }

    @Override
    public double getPaymentAmount() {
        return getQuantity() * getPricePerItem();
    }
}
```

Output :

```
========================================
1. PENGUJIAN ABSTRACT CLASS (EMPLOYEE)
========================================
salaried employee: John Smith
social security number: 111-11-1111
weekly salary: $800,00
Total Earnings: $800,00

========================================
2. PENGUJIAN INTERFACE (PAYABLE & INVOICE)
========================================
invoice:
part number: 01234 (Seat)
quantity: 2
price per item: $375,00
Total Payment: $750,00
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

3. Konstanta Interface

Product

```
package com.mycompany.guidedmodul07;

public interface Product {
    static final String MAKER = "My Corp";
    static final String PHONE = "555-123-4567";

    int getPrice(int id);
}
```

Shoe

```
package com.mycompany.guidedmodul07;

public class Shoe implements Product {
    @Override
    public int getPrice(int id) {
        if (id == 1) {
            return 5;
        } else {
            return 10;
        }
    }

    public String getMaker() {
        return MAKER;
```

```
    }
}
```

Store

```
package com.mycompany.guidedmodul07;

public class Store {
    static Shoe hightop;

    public static void init() {
        hightop = new Shoe();
    }

    public static void getInfo(Shoe item) {
        System.out.println("This Product is made by " + item.getMaker());
        System.out.println("It costs $" + item.getPrice(1) + "\n");
    }

    public static void orderInfo(Product item) {
        System.out.println("To order from " + Product.MAKER + " call " + Product.PHONE
+ ".");
        System.out.println("Each item costs $" + item.getPrice(1));
    }

    public static void main(String[] args) {
        init();
        getInfo(hightop);
        orderInfo(hightop);
    }
}
```

Output :

```
This Product is made by My Corp
It costs $5

To order from My Corp call 555-123-4567.
- Each item costs $5
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

4. Fitur Java 8

MyInterface

```
package com.mycompany.guidedmodul07;
```

```
public interface MyInterface {
    default void newMethod() {
        System.out.println("Newly added default method");
    }

    static void anotherNewMethod() {
        System.out.println("Newly added static method");
    }

    void existingMethod(String str);
}
```

Example

```
package com.mycompany.guidedmodul07;

public class Example implements MyInterface {
    @Override
    public void existingMethod(String str) {
        System.out.println("String is: " + str);
    }

    public static void main(String[] args) {
        Example obj = new Example();
        obj.newMethod();
        MyInterface.anotherNewMethod();
        obj.existingMethod("Java 8 is easy to learn");
    }
}
```

Output :

```
Newly added default method
Newly added static method
String is: Java 8 is easy to learn
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

C. Unguided

1. Soal Unguided 1

Sebuah bioskop memerlukan sistem kasir yang mampu memproses pembayaran dalam satu keranjang belanja. Di kasir, penonton tidak hanya membeli **Tiket Nonton**, tetapi juga bisa membeli makanan/minuman (**Paket Snack F&B**).

Secara konsep sistem, **Tiket Bioskop** dan **Paket Snack** berasal dari silsilah keluarga (hierarki) yang sepenuhnya berbeda. Tiket film murni pun bersifat abstrak (tidak bisa dibeli tanpa jenis studio). Namun, di mata program kasir, tiket dan makanan harus bisa disatukan, dihitung total tagihannya, dan dicetak di dalam satu struk secara otomatis.

Pengerjaan soal ini dibagi menjadi 4 tugas utama untuk melatih abstraksi kelas, antarmuka (*interface*), dan fitur modern Java 8+.

Jawaban :

ItemTransaksi

```
package com.mycompany.unguidedmodul07;

public interface ItemTransaksi {
    // Kontrak metode abstrak
    double getNominalBayar();
    String getNamaTransaksi();

    // Default method Java 8+ untuk mencetak baris rincian item
    default void cetakRincianItem() {
        System.out.println(getNamaTransaksi() + " | Total: Rp" +
getNominalBayar());
    }

    // Static method Java 8+ untuk pencatatan log sistem kasir
    static void catatLogKasir(String pesan) {
        System.out.println("[LOG SISTEM KASIR] " + pesan);
    }
}
```

TiketBioskop

```
package com.mycompany.unguidedmodul07;

public abstract class TiketBioskop implements ItemTransaksi {
    private String judulFilm;
    private double hargaDasar;
    private int jumlahKursi;

    // Konstruktor dengan kata kunci this eksplisit
    public TiketBioskop(String judulFilm, double hargaDasar, int jumlahKursi) {
        this.judulFilm = judulFilm;
        this.hargaDasar = hargaDasar;
        this.jumlahKursi = jumlahKursi;
    }

    public String getJudulFilm() {
        return this.judulFilm;
    }
```

```
    public double getHargaDasar() {
        return this.hargaDasar;
    }

    public int getJumlahKursi() {
        return this.jumlahKursi;
    }

    // Abstract method internal spesifik tiket
    public abstract double hitungTotalBayar();

    @Override
    public abstract String getNamaTransaksi();

    // Mewujudkan kontrak antarmuka ItemTransaksi dengan memanggil
hitungTotalBayar()
    @Override
    public double getNominalBayar() {
        return hitungTotalBayar();
    }
}
```

TiketReguler

```
package com.mycompany.unguidedmodul07;

public class TiketReguler extends TiketBioskop {
    private int nomorStudio;

    public TiketReguler(String judulFilm, double hargaDasar, int jumlahKursi, int
nomorStudio) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.nomorStudio = nomorStudio;
    }

    public int getNomorStudio() {
        return this.nomorStudio;
    }

    @Override
    public double hitungTotalBayar() {
        return getHargaDasar() * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket Reguler (Studio " + this.nomorStudio + ") - " + getJudulFilm()
                + " (" + getJumlahKursi() + " x Rp" + getHargaDasar() + ")";
```

```
        }
}
```

TiketVIP

```
package com.mycompany.unguidedmodul07;

public class TiketVIP extends TiketBioskop {
    private double biayaFasilitasVIP;

    public TiketVIP(String judulFilm, double hargaDasar, int jumlahKursi, double
biayaFasilitasVIP) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.biayaFasilitasVIP = biayaFasilitasVIP;
    }

    public double getBiayaFasilitasVIP() {
        return this.biayaFasilitasVIP;
    }

    @Override
    public double hitungTotalBayar() {
        return (getHargaDasar() + this.biayaFasilitasVIP) * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        double totalPerKursi = getHargaDasar() + this.biayaFasilitasVIP;
        return "Tiket VIP Premiere - " + getJudulFilm()
                + " (" + getJumlahKursi() + " x Rp" + totalPerKursi + ")";
    }
}
```

PaketSnack

```
package com.mycompany.unguidedmodul07;

public class PaketSnack implements ItemTransaksi {
    private String namaPaket;
    private int jumlahPorsi;
    private double hargaPerPorsi;

    public PaketSnack(String namaPaket, int jumlahPorsi, double hargaPerPorsi) {
        this.namaPaket = namaPaket;
        this.jumlahPorsi = jumlahPorsi;
        this.hargaPerPorsi = hargaPerPorsi;
    }

    public String getNamaPaket() {
        return this.namaPaket;
```

```
    }

    public int getJumlahPorsi() {
        return this.jumlahPorsi;
    }

    public double getHargaPerPorsi() {
        return this.hargaPerPorsi;
    }

    @Override
    public double getNominalBayar() {
        return this.jumlahPorsi * this.hargaPerPorsi;
    }

    @Override
    public String getNamaTransaksi() {
        return "F&B Snack: " + this.namaPaket
                + " (" + this.jumlahPorsi + " x Rp" + this.hargaPerPorsi + ")";
    }
}
```

Main

```
package com.mycompany.unguidedmodul07;

public class Main {
    public static void main(String[] args) {
        // 1. Uji Proteksi Abstraksi: Kelas abstrak tidak dapat diinstansiasi secara langsung
        // TiketBioskop ujiAbstrak = new TiketBioskop("Avatar", 50000, 1);

        // 2. Buka Kasir melalui static method antarmuka
        ItemTransaksi.catatLogKasir("Membuka transaksi baru untuk Pelanggan #042.");

        // 3. Buat Objek Transaksi sesuai parameter soal
        TiketReguler tiket1 = new TiketReguler("Interstellar", 40000.0, 2, 3);
        TiketVIP tiket2 = new TiketVIP("Dune: Part Two", 60000.0, 2, 25000.0);
        PaketSnack snack1 = new PaketSnack("Combo Popcorn Caramel + 2 Soda", 1,
55000.0);

        // 4. Array Polimorfik Heterogen (Array statis standar)
        ItemTransaksi[] daftarPesanan = new ItemTransaksi[3];
        daftarPesanan[0] = tiket1;
        daftarPesanan[1] = tiket2;
        daftarPesanan[2] = snack1;

        // 5. Pemrosesan Dinamis
        System.out.println("\n=== DAFTAR TRANSAKSI BIOSKOP ===");
        double grandTotal = 0.0;
```

```
        for (ItemTransaksi item : daftarPesanan) {
            item.cetakRincianItem(); // Memanggil default method
            grandTotal += item.getNominalBayar(); // Akumulasi total belanja
        }

        // 6. Tutup Kasir
        System.out.println("-------------------------------------------------------");
        System.out.println("TOTAL TRANSAKSI: Rp" + grandTotal);

System.out.println("=======================================================
====");
        ItemTransaksi.catatLogKasir("Transaksi #042 selesai diproses.");
    }
}
```

Output :

```
[LOG SISTEM KASIR] Membuka transaksi baru untuk Pelanggan #042.

=== DAFTAR TRANSAKSI BIOSKOP ===
Tiket Reguler (Studio 3) - Interstellar (2 x Rp40000.0) | Total: Rp80000.0
Tiket VIP Premiere - Dune: Part Two (2 x Rp85000.0) | Total: Rp170000.0
F&B Snack: Combo Popcorn Caramel + 2 Soda (1 x Rp55000.0) | Total: Rp55000.0
-------------------------------------------------------
TOTAL TRANSAKSI: Rp305000.0
=======================================================
[LOG SISTEM KASIR] Transaksi #042 selesai diproses.
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Penjelasan :

Program kasir bioskop ini menerapkan konsep abstraksi, kontrak antarmuka, dan polimorfisme dengan menyatukan dua entitas berbeda—yaitu tiket dan makanan—ke dalam satu antarmuka ItemTransaksi yang memuat kontrak method perhitungan, *default method* cetakRincianItem() untuk pencetakan struk, serta *static method* catatLogKasir() untuk pencatatan log. Kelas abstrak TiketBioskop berperan sebagai fondasi yang mengamankan data tiket melalui enkapsulasi dan kata kunci this, sekaligus mendelegasikan perumusan logika biaya ke kelas turunan konkretnya, yaitu TiketReguler (tarif dasar dikali kuantitas kursi) dan TiketVIP (penambahan biaya fasilitas per kursi). Di sisi lain, kelas independen PaketSnack turut mengimplementasikan kontrak antarmuka tersebut tanpa mewarisi hierarki tiket. Seluruh objek transaksi tersebut kemudian dihimpun secara heterogen ke dalam array statis bertipe ItemTransaksi[] pada kelas Main, sehingga program dapat mengeksekusi method pencetakan dan mengakumulasi total pembayaran akhir secara

dinamis dan seragam melalui perulangan tanpa perlu membedakan tipe spesifik masing-masing objek.

D. Kesimpulan

Praktikum ini membuktikan bahwa kelas abstrak (*abstract class*) dan antarmuka (*interface*) merupakan mekanisme penting dalam Pemrograman Berorientasi Objek untuk membangun sistem perangkat lunak yang terstruktur, modular, dan fleksibel. Kelas abstrak efektif digunakan sebagai kerangka induk bagi entitas-entitas yang memiliki relasi keturunan langsung (*inheritance*) guna mewariskan properti umum sekaligus memaksakan implementasi *method* spesifik pada *subclass*-nya. Sementara itu, *interface* berperan sebagai kontrak perilaku yang memungkinkan terjalinnya polimorfisme heterogen lintas entitas yang tidak saling berkaitan hierarki—seperti menyatukan transaksi tiket dan makanan dalam satu antarmuka kasir tanpa ikatan pewarisan. Ditambah dengan fitur modern Java 8 berupa *default method* dan *static method*, antarmuka kini dapat menyediakan fungsionalitas bersama dan utilitas sistem secara efisien tanpa merusak kompatibilitas kelas-kelas pengimplementasinya.